



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

ROUTING GUIDE

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	ROUTING GUIDE
Version	v1.0.0
Date	02 July 2026

React Router — Complete Navigation System

1. What is Client-Side Routing?

Traditional websites: clicking a link requests a new HTML page from the server (full page reload).

Single Page Applications (SPAs) like this one: the browser loads one HTML file ([index.html](#)) and JavaScript handles all navigation by swapping components — no page reload.

React Router DOM provides this SPA routing.

2. Complete Route Tree

/	→ HomePage	(MainLayout)
/login	→ LoginPage	(AuthLayout + PublicRoute)
/register	→ RegisterPage	(AuthLayout + PublicRoute)
/verify-otp	→ VerifyOtpPage	(AuthLayout + PublicRoute)
/forgot-password	→ ForgotPasswordPage	(AuthLayout + PublicRoute)
/reset-password	→ ResetPasswordPage	(AuthLayout + PublicRoute)
/dashboard	→ UserDashboard	(UserLayout + ProtectedRoute[patient,doctor])
/search	→ MedicineSearchPage	(UserLayout + ProtectedRoute)
/search/results	→ SearchResultsPage	(UserLayout + ProtectedRoute)
/medicine/:id	→ MedicineDetailsPage	(UserLayout + ProtectedRoute)
/medicine/:id/generic	→ GenericRecommendationPage	(UserLayout + ProtectedRoute)
/pharmacies/nearby	→ NearbyPharmaciesPage	(UserLayout + ProtectedRoute)
/notifications	→ NotificationsPage	(UserLayout + ProtectedRoute)
/profile	→ ProfilePage	(UserLayout + ProtectedRoute)
/pharmacy/dashboard	→ PharmacyDashboard	(PharmacyLayout + ProtectedRoute[pharmacist])
/pharmacy/inventory	→ InventoryPage	(PharmacyLayout + ProtectedRoute)
/pharmacy/inventory/add	→ MedicineFormPage	(PharmacyLayout + ProtectedRoute)
/pharmacy/inventory/edit/:id	→ MedicineFormPage	(PharmacyLayout + ProtectedRoute)
/pharmacy/prescriptions	→ InventoryPage*	(PharmacyLayout + ProtectedRoute)
/pharmacy/profile	→ ProfilePage	(PharmacyLayout + ProtectedRoute)
/admin/dashboard	→ AdminDashboard	(AdminLayout + ProtectedRoute[admin])
/admin/users	→ AdminUsers	(AdminLayout + ProtectedRoute)
/admin/pharmacies	→ AdminPharmacies	(AdminLayout + ProtectedRoute)
/admin/medicines	→ AdminMedicines	(AdminLayout + ProtectedRoute)
/admin/generic-mapping	→ AdminGenericMapping	(AdminLayout + ProtectedRoute)
/admin/analytics	→ AdminAnalytics	(AdminLayout + ProtectedRoute)
/admin/reports	→ AdminReports	(AdminLayout + ProtectedRoute)
/admin/notifications	→ AdminNotifications	(AdminLayout + ProtectedRoute)
/admin/activity	→ AdminActivity	(AdminLayout + ProtectedRoute)
/admin/roles	→ AdminRoles	(AdminLayout + ProtectedRoute)
/admin/settings	→ AdminSettings	(AdminLayout + ProtectedRoute)
/unauthorized	→ UnauthorizedPage	(standalone)
/session-expired	→ SessionExpiredPage	(standalone)
/*	→ NotFoundPage	(catch-all)

* Prescriptions uses InventoryPage as a placeholder until that module is built

3. Layouts as Route Wrappers

React Router's `<Outlet />` pattern allows layouts to wrap page content:

```
// UserLayout renders once and page content changes inside it
function UserLayout() {
  return (
    <div className="flex h-screen">
      <Sidebar />
      <div className="flex flex-col flex-1">
        <TopBar />
        <main className="page-enter flex-1 overflow-y-auto p-6">
          <Outlet /> { /* Page renders here */ }
        </main>
      </div>
    </div>
  )
}
```

In `AppRouter.jsx`:

```
<Route element={<ProtectedRoute allowedRoles={[USER_ROLES.PATIENT, USER_ROLES.DOCTOR]} />}>
  <Route element={<UserLayout />}>
    <Route path="dashboard" element={<UserDashboard />} /> { /* → Outlet */ }
    <Route path="search" element={<MedicineSearchPage />} /> { /* → Outlet */ }
  </Route>
</Route>
```

When the user visits `/dashboard`:

1. `ProtectedRoute` checks auth → passes
2. `UserLayout` renders (Sidebar + TopBar + Outlet)
3. `UserDashboard` renders inside Outlet

4. ProtectedRoute — Authentication Guard

```
function ProtectedRoute({ allowedRoles }) {
  const { isAuthenticated, currentUser, isLoading } = useAuth()
  const location = useLocation()

  // During auth rehydration – show spinner, not redirect
  if (isLoading) return <Spinner />

  // Not logged in → go to login, remember where they were
  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />
  }

  // Logged in but wrong role → 403 page
  if (allowedRoles && !allowedRoles.includes(currentUser?.role)) {
    return <Navigate to="/unauthorized" replace />
  }

  // All checks passed → render the actual page
  return <Outlet />
}
```

The `state={{ from: location }}` pattern:

After login, the app can redirect back to where the user was trying to go:

```
// In LoginPage after successful login:
const from = location.state?.from?.pathname || ROUTES.USER.DASHBOARD
navigate(from, { replace: true })
```

5. PublicRoute — Redirect Authenticated Users

Prevents logged-in users from accessing auth pages (e.g. visiting /login while already logged in):

```
function PublicRoute() {
  const { isAuthenticated, currentUser } = useAuth()

  if (isAuthenticated) {
    // Role-based redirect
    switch (currentUser?.role) {
      case 'admin':      return <Navigate to="/admin/dashboard" replace />
      case 'pharmacist': return <Navigate to="/pharmacy/dashboard" replace />
      default:           return <Navigate to="/dashboard" replace />
    }
  }

  return <Outlet /> // Not authenticated → show auth page
}
```

6. Navigation Flow Diagram

```

User visits /dashboard
  ↓
ProtectedRoute checks isAuthenticated
  ↓
[No] → Navigate to /login
[Yes] → Check allowedRoles
      ↓
      [Role matches] → Render UserLayout → Render UserDashboard
      [Role mismatch] → Navigate to /unauthorized

User visits /login (already logged in)
  ↓
PublicRoute checks isAuthenticated
  ↓
[Yes] → Navigate to role dashboard
[No] → Render AuthLayout → Render LoginPage
  
```

7. URL Parameters

Dynamic routes use `:paramName` syntax:

```
<Route path="medicine/:id"          element={<MedicineDetailsPage />} />
<Route path="pharmacy/inventory/edit/:id" element={<MedicineFormPage />} />
```

Reading the parameter in the page:

```
import { useParams } from 'react-router-dom'

function MedicineDetailsPage() {
  const { id } = useParams()
  // id = "123" when URL is /medicine/123
  // TODO: fetch medicine data with medicineService.getById(id)
}
```

8. Programmatic Navigation

Using `useNavigate()` to navigate from JavaScript code (not a `<Link>` click):

```
import { useNavigate } from 'react-router-dom'

function SearchResultCard({ medicine }) {
  const navigate = useNavigate()

  return (
    <button onClick={() => navigate(`/medicine/${medicine.id}`)}>
      View Details
    </button>
  )
}
```

Passing state to the next page:

```
navigate('/verify-otp', { state: { email: formData.email } })

// In VerifyOtpPage:
const location = useLocation()
const email = location.state?.email
```

9. Lazy Loading Routes

All page-level components are lazily loaded:

```
// Normal import – loads immediately (auth pages – small and critical)
import LoginPage from '../pages/auth/LoginPage'

// Lazy import – loads only when user navigates to that route
const UserDashboard = lazy(() => import('../pages/dashboard/UserDashboard'))
const AdminDashboard = lazy(() => import('../pages/admin/AdminDashboard'))
```

`<Suspense>` provides a fallback while the lazy component loads:

```
<Suspense fallback={<PageLoader />}>
  <Routes>
    {/* all routes here */}
  </Routes>
</Suspense>
```

Result: Main bundle is 464KB instead of 963KB — 50% faster initial load.

10. Route Constants

All route paths are stored in `constants/routes.js` to prevent typos:

```
// ■ Safe – typos caught at import time
import { ROUTES } from '../constants/routes'
navigate(ROUTES.USER.DASHBOARD)

// ■ Risky – typos discovered at runtime
navigate('/dahsboard') // typo – would silently 404

export const ROUTES = {
  USER: {
    DASHBOARD: '/dashboard',
    SEARCH: '/search',
    SEARCH_RESULTS: '/search/results',
    MEDICINE_DETAIL: '/medicine/:id',
    NEARBY_PHARMACIES: '/pharmacies/nearby',
  },
  ADMIN: {
    DASHBOARD: '/admin/dashboard',
    USERS: '/admin/users',
  },
  SESSION_EXPIRED: '/session-expired',
}
```

11. Breadcrumb Navigation

The Breadcrumb component shows the hierarchy:

Home > Medicines > Paracetamol 500mg

Used in pages to help users understand where they are and navigate back:

```
<Breadcrumb items={[
  { label: 'Search', to: ROUTES.USER.SEARCH },
  { label: 'Results', to: ROUTES.USER.SEARCH_RESULTS },
  { label: medicine.name } // last item has no 'to' – it's current page
]} />
```